Probability Estimation via Monte Carlo Methods: The Adjacent Kings Experiment

A computational approach to combinatorics

1. Introduction

The Monte Carlo method is a powerful statistical technique that uses random sampling to obtain numerical results. In this project, taking inspiration from the video https://www.youtube.com/watch?v=SGwUUTTw63g, we applied this method to solve a classical probability problem: determining the frequency with which at least two Kings appear in adjacent positions in a randomly shuffled deck of 52 cards. While the analytical calculation can be complex, the Monte Carlo simulation allows us to approximate the solution by "forcing" the problem through thousands of repeated trials.

2. Theoretical and Analytical Background

To understand the validity of the simulation, it is fundamental to compare it with the exact mathematical solution. The problem asks for the probability that there are at least two adjacent Kings.

2.1 The Analytical Formula

The simplest way to solve the problem is to calculate the probability of the complementary event, i.e., that *no* King is adjacent to another.

1. Remove the 4 Kings from the deck, leaving 48 non-King cards.
2. These 48 cards create 49 possible slots (the spaces between one card and another, plus the ends) where the Kings can be inserted without being adjacent.
3. The number of ways to choose 4 positions out of 49 is given by the binomial coefficient $\binom{49}{4}$.
4. The total number of ways to position 4 Kings in 52 cards is $\binom{52}{4}$.

The final formula for the probability is:

$$P = 1 - \frac{\binom{49}{4}}{\binom{52}{4}} \approx 0.217376 \text{ (or approximately } 21.74\%)$$

3. The Original Prompt

The basis for this section of the video was generated by artificial intelligence, specifically NotebookLM, based on this prompt:

> "Hi, could you make me a video with these specifications? The style of the whole video must be hand-drawn drawings. Subject: an explanation of the Monte Carlo simulation method with a deck of cards. First, you calculate analytically the probability of having two adjacent kings, then you show a sequence of images where you simulate the same

task many times. At that point, I will externally add a Python script that does exactly this. This must be your final part: I will complete it later by adding the numbers, the graph, and the script."

4. Python Simulation

To support the video, a Python code was developed that replicates the shuffling and checking process thousands of times. The simulation generates a visualization of the convergence towards the analytical probability.

```
import random
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from math import comb

def check_adjacent_kings(deck):
    suits = ['Hearts', 'Diamonds', 'Clubs', 'Spades']
    ranks = ['2', '3', '4', '5', '6', '7', '8', '9', '10', 'J', 'Q', 'K', 'A']
    sim_deck = ['K' if rank == 'K' else 'non_K' for rank in ranks for _ in suits]

    random.shuffle(sim_deck)

    for i in range(len(sim_deck) - 1):
        if sim_deck[i] == 'K' and sim_deck[i+1] == 'K':
            return True
    return False

def run_monte_carlo_simulation(num_simulations):
    results = []
    hits = 0
    points_to_plot = min(500, num_simulations)
    step = max(1, num_simulations // points_to_plot)

    for i in range(1, num_simulations + 1):
        if check_adjacent_kings(None):
            hits += 1

        if i % step == 0 or i == num_simulations:
            estimated_probability = hits / i
            results.append((i, estimated_probability))

    return np.array(results)

def calculate_analytical_probability():
    total_arrangements = comb(52, 4)
    non_adjacent_arrangements = comb(49, 4)

    prob_no_adjacent = non_adjacent_arrangements / total_arrangements
    prob_at_least_one_adjacent = 1 - prob_no_adjacent
    return prob_at_least_one_adjacent

num_simulations = 200000
print(f"Running {num_simulations} Monte Carlo simulations...")
monte_carlo_results = run_monte_carlo_simulation(num_simulations)

analytical_prob = calculate_analytical_probability()
print(f"Analytical probability of adjacent Kings: {analytical_prob:.6f}")
```

```
print(f"Final Monte Carlo estimate: {monte_carlo_results[-1, 1]:.6f}")

plt.style.use('seaborn-v0_8-darkgrid')
plt.figure(figsize=(12, 7))

plt.plot(monte_carlo_results[:, 0], monte_carlo_results[:, 1],
         color='#1f77b4',
         linewidth=2.5,
         label='Monte Carlo Estimate')

plt.axhline(y=analytical_prob,
            color='red',
            linestyle='--',
            linewidth=2,
            label=f'Analytical Probability ({analytical_prob:.4f})')

final_x = monte_carlo_results[-1, 0]
final_y = monte_carlo_results[-1, 1]
plt.scatter(final_x, final_y, color='red', s=100, zorder=5)
plt.annotate(f'{final_y:.4f}',
             (final_x, final_y),
             textcoords="offset points",
             xytext=(0,10),
             ha='center',
             color='red',
             fontsize=10,
             weight='bold')

plt.title(f'Monte Carlo Estimate Convergence ({num_simulations:,} Simulations)',
          fontsize=18,
          weight='bold',
          color='#333333')
plt.xlabel('Number of Simulations',
           fontsize=14,
           color='#555555')
plt.ylabel('Estimated Probability of Adjacent Kings',
           fontsize=14,
           color='#555555')

plt.xscale('log')
plt.xlim(10, num_simulations * 1.1)
plt.ylim(0.15, 0.25)

plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(fontsize=12)
plt.tick_params(labelsize=11, colors='#555555')

plt.tight_layout()
plt.show()
```

5. Results and Convergence

The simulation demonstrates the Law of Large Numbers: as the number of trials (n) increases, the estimated probability converges towards the analytical value of 0.2173.

- **After 10,000 simulations:** The estimate might fluctuate significantly.
- **After 200,000 simulations:** The result stabilizes with a very low percentage error.

- **Visualization:** The graph generated by the code (using `plt.xscale('log')`) clearly shows how the simulation line "flattens" onto the red dashed line of the analytical probability as the attempts increase.

For transparency, this report was drafted in Italian and then automatically translated using artificial intelligence tools.